

به نام خدا



پروژه اول درس مبانی هوش مصنوعی

ابراهیم کیوانی 4023623024

گزارش پروژه اول

حل مسئله Sokoban با الگوریتم‌های جستجو

1. مقدمه

مسئله Sokoban یکی از مسائل کلاسیک در حوزه‌ی جستجو در فضای حالت است که در آن عامل باید با حرکت در یک محیط شبکه‌ای، همه‌ی جعبه‌ها را به خانه‌های هدف مشخص شده برساند. این مسئله به دلیل ترکیب محدودیت‌های حرکتی، وجود موانع ثابت، و نیاز به برنامه‌ریزی چندمرحله‌ای، نمونه‌ی مناسبی برای بررسی الگوریتم‌های جستجوی آگاهانه و ناآگاهانه به شمار می‌رود.

در این پروژه، هدف تنها یافتن یک مسیر معتبر نیست، بلکه پیدا کردن مسیر بهینه نیز اهمیت دارد. به همین دلیل، علاوه بر درستی جواب، معیار دیگری نیز برای ارزیابی عملکرد الگوریتم‌ها در نظر گرفته شده است: تعداد گره‌های بسطیافته یا Expanded Nodes. این معیار نشان می‌دهد هر الگوریتم برای رسیدن به جواب، چه مقدار از فضای حالت را بررسی کرده است.

در مدل هزینه‌ی این پروژه، همه‌ی اعمال یکسان در نظر گرفته نشده‌اند.

- حرکت معمولی بازیکن دارای هزینه‌ی 1 است.
- هل دادن هر جعبه دارای هزینه‌ی 5 است.

این موضوع باعث می‌شود که کوتاه‌ترین مسیر از نظر تعداد حرکت، الزاماً کم‌هزینه‌ترین مسیر نباشد. بنابراین الگوریتمی مانند UCS یا A^* که هزینه‌ی واقعی مسیر را در نظر می‌گیرند، نسبت به BFS اهمیت بیشتری پیدا می‌کنند.

در این گزارش، ابتدا مسئله مدل‌سازی می‌شود، سپس تکنیک‌های بهینه‌سازی فضای حالت و تشخیص بن‌بست‌ها بررسی می‌شوند، و در ادامه عملکرد الگوریتم‌های BFS، IDS، UCS، و A^* در حل Sokoban تحلیل می‌شود. در پایان نیز نتایج تجربی با توجه به سطح سختی نقشه‌ها و معیار Expanded Nodes جمع‌بندی خواهد شد.

2. مدل سازی مسئله

برای حل Sokoban با روش های جستجو، ابتدا باید مسئله را به صورت یک مسئله ی جستجو در فضای حالت تعریف کنیم. در این مدل، هر حالت نمایانگر وضعیت کامل بازی در یک لحظه ی مشخص است.

2.1 اجزای حالت

هر حالت معمولاً شامل موارد زیر است:

- موقعیت فعلی بازیکن
- موقعیت تمام جعبه ها
- وضعیت خانه های هدف
- نقشه ی ثابت شامل دیوارها و موانع

در پیاده سازی پروژه، برای اینکه حالت ها قابل مقایسه و ذخیره سازی باشند، موقعیت جعبه ها معمولاً به شکل ساختارهای غیر قابل تغییر مانند tuple نگهداری می شود تا بتوان از آن ها به عنوان کلید در مجموعه ها و دیکشنری ها استفاده کرد.

2.2 عملگرها

در Sokoban بازیکن می تواند در چهار جهت اصلی حرکت کند:

- بالا
- پایین
- چپ
- راست

اگر خانه ی مقصد خالی باشد، حرکت عادی انجام می شود.

اگر خانه ی مقصد شامل جعبه باشد و خانه ی پشت آن نیز خالی باشد، جعبه هل داده می شود.

2.3 تابع هدف

هدف زمانی محقق می شود که تمام جعبه ها روی خانه های هدف قرار گرفته باشند.

بنابراین، شرط نهایی فقط رسیدن بازیکن به یک نقطه ی خاص نیست، بلکه آرایش جعبه ها در محیط باید به وضعیت مطلوب برسد.

2.4 هزینه مسیر

در این پروژه، هزینه ی مسیر به صورت زیر تعریف می شود:

$$\text{Cost} = (\text{number of normal moves}) \times 1 + (\text{number of box pushes}) \times 5$$
$$\text{Cost} = (\text{number of normal moves}) \times 1 + (\text{number of box pushes}) \times 5$$

این تعریف باعث می‌شود الگوریتم‌هایی که بر اساس هزینه‌ی تجمعی مسیر عمل می‌کنند، مانند UCS و A^* ، بتوانند راحل‌هایی با هزینه‌ی کمتر پیدا کنند؛ حتی اگر آن مسیر از نظر تعداد گام طولانی‌تر باشد.

3. بهینه‌سازی فضای حالت و تشخیص بن‌بست‌ها

در مسائل جستجو، یکی از بزرگ‌ترین چالش‌ها رشد سریع فضای حالت است. این مشکل در Sokoban شدیدتر نیز هست، زیرا تعداد حالت‌های ممکن با افزایش تعداد جعبه‌ها و اندازه‌ی نقشه به‌صورت نمایی رشد می‌کند. به همین دلیل، پیش از اعمال الگوریتم جستجو باید تا حد امکان حالت‌های نامعتبر یا بی‌فایده حذف شوند.

3.1 اهمیت pruning

اگر همه‌ی حالت‌ها بدون فیلتر بررسی شوند، الگوریتم‌ها خیلی زود با انفجار فضای حالت مواجه می‌شوند. به‌خصوص در BFS و IDS، تعداد گره‌های تولیدی می‌تواند بسیار زیاد شود و حافظه یا زمان اجرای برنامه را تحت فشار قرار دهد. بنابراین استفاده از pruning ضروری است.

3.2 بن‌بست‌های ایستا

یکی از مهم‌ترین روش‌های کاهش فضای جستجو، تشخیص deadlockهاست.

بن‌بست زمانی رخ می‌دهد که جعبه‌ای در موقعیتی قرار گیرد که دیگر هیچ‌گاه نتوان آن را به هدف رساند.

نمونه‌های رایج بن‌بست ایستا:

- قرار گرفتن جعبه در گوشه‌ای غیر از هدف
- گیر افتادن جعبه بین دو دیوار
- قرار گرفتن جعبه در محلی که از هیچ سمت قابل انتقال نیست

این نوع بن‌بست‌ها به وضعیت ثابت نقشه وابسته‌اند و معمولاً مستقل از موقعیت سایر جعبه‌ها هستند.

3.3 بن‌بست‌های مربعی یا پویای محلی

علاوه بر deadlockهای ایستا، بعضی آرایش‌های جعبه‌ها نیز به شکست منجر می‌شوند.

برای مثال، اگر دو جعبه به شکلی در یک فضای محدود قرار بگیرند که عملاً مانع حرکت یکدیگر شوند، وضعیت حاصل غیرقابل حل خواهد بود.

در پروژه‌ی شما، استفاده از تشخیص این نوع بن‌بست‌ها باعث شده تعداد زیادی از حالت‌های بی‌فایده پیش از ورود به فضای اصلی جستجو حذف شوند. این موضوع به‌ویژه در A^* اثر زیادی بر کاهش Expanded Nodes دارد.

3.4 اثر deadlock detection بر عملکرد

تشخیص بن‌بست‌ها دو فایده‌ی مهم دارد:

1. از گسترش حالت‌های بی‌نتیجه جلوگیری می‌کند.

2. باعث می‌شود الگوریتم‌ها زمان و حافظه‌ی کمتری مصرف کنند.

در نتیجه، حتی اگر خود الگوریتم جستجو تغییر نکند، با pruning بهتر می‌توان عملکرد آن را به‌طور قابل‌توجهی بهبود داد.

4. الگوریتم BFS

4.1 ایده‌ی کلی

BFS یا جستجوی سطح‌به‌سطح، یکی از ساده‌ترین الگوریتم‌های جستجوی ناآگاهانه است. این الگوریتم ابتدا همه‌ی حالت‌های عمق 0، سپس عمق 1، بعد عمق 2 و به همین ترتیب را بررسی می‌کند.

ویژگی اصلی BFS این است که اگر همه‌ی هزینه‌ها برابر باشند، می‌تواند کوتاه‌ترین مسیر از نظر تعداد گام را پیدا کند. اما در مسئله‌ی Sokoban، به دلیل متفاوت بودن هزینه‌ی حرکت و هل دادن جعبه، BFS لزوماً کم‌هزینه‌ترین جواب را نمی‌دهد.

4.2 پیاده‌سازی

در پیاده‌سازی پروژه، BFS از یک صف برای نگهداری حالات مرزی استفاده می‌کند. هر بار اولین حالت از صف برداشته شده و فرزندان آن تولید می‌شوند. برای جلوگیری از تکرار، حالت‌های دیده‌شده در ساختاری مانند set ذخیره می‌شوند.

4.3 مزایا

- ساده و قابل‌فهم
- اگر هزینه‌ها برابر باشند، جواب بهینه از نظر تعداد گام می‌دهد
- تضمین کامل بودن دارد

4.4 محدودیت‌ها در Sokoban

BFS در Sokoban معمولاً عملکرد ضعیفی دارد، زیرا:

- تعداد حالت‌ها بسیار زیاد است
- حافظه‌ی زیادی مصرف می‌کند
- به هزینه‌ی واقعی مسیر توجه نمی‌کند
- در نقشه‌های متوسط و سخت ممکن است انفجار فضای حالت رخ دهد

بنابراین BFS بیشتر برای نقشه‌های ساده مناسب است.

5. الگوریتم IDS

5.1 ایده‌ی کلی

IDS یا جستجوی عمق محدود تکرار شونده، ترکیبی از مزایای BFS و DFS است. در این الگوریتم، ابتدا جستجو تا عمق 0 انجام می‌شود، سپس تا عمق 1، بعد 2 و همین‌طور ادامه می‌یابد تا جواب پیدا شود.

5.2 پیاده‌سازی

IDS در هر مرحله یک DFS با محدودیت عمق اجرا می‌کند. اگر جواب در آن عمق پیدا نشود، عمق افزایش می‌یابد. این روش باعث می‌شود در عین حفظ کامل بودن، مصرف حافظه کمتر از BFS باشد.

5.3 مزایا

- حافظه‌ی کمتر نسبت به BFS
- کامل بودن در فضای محدود
- مناسب برای مسائلی که عمق جواب مشخص نیست

5.4 محدودیت‌ها در Sokoban

با وجود مزیت حافظه‌ای، IDS در Sokoban چند مشکل دارد:

- تکرار زیاد جستجو در هر عمق
- بررسی دوباره‌ی بسیاری از حالت‌ها
- حساسیت بالا به pruning
- عملکرد ضعیف در نقشه‌های بزرگتر

به همین دلیل، IDS معمولاً از نظر Expanded Nodes از BFS هم می‌تواند ضعیف‌تر شود، مخصوصاً اگر فضای حالت به‌خوبی هرس نشده باشد.

6. الگوریتم UCS

6.1 ایده‌ی کلی

UCS یا Uniform Cost Search، الگوریتمی است که در هر مرحله حالتی را گسترش می‌دهد که کمترین هزینه‌ی تجمعی را تا آن لحظه داشته باشد. این الگوریتم نسبت به BFS پیشرفته‌تر است، زیرا به‌جای عمق، هزینه را معیار انتخاب قرار می‌دهد.

6.2 اهمیت UCS در این پروژه

از آنجا که در Sokoban هزینه‌ی حرکت عادی و هل دادن جعبه متفاوت است، UCS می‌تواند مسیرهایی را پیدا کند که واقعاً کم‌هزینه‌تر هستند. بنابراین UCS نسبت به BFS انتخاب منطقی‌تری برای این مسئله است.

6.3 پیاده‌سازی

UCS معمولاً با صف اولویت‌دار پیاده‌سازی می‌شود. هر حالت همراه با $g(n)$ یعنی هزینه‌ی طی‌شده از ابتدا تا آن حالت ذخیره می‌شود. حالتی که کمترین g را دارد، ابتدا گسترش می‌یابد.

6.4 مزایا

- یافتن مسیر کم هزینه
- کامل بودن
- مناسب برای هزینه‌های ناهمسان

6.5 محدودیت‌ها

- در حالت بدون heuristic، ممکن است تعداد زیادی گره گسترش یابد
- در نقشه‌های سخت، زمان اجرا می‌تواند زیاد شود
- بدون deadlock pruning هنوز با انفجار فضای حالت مواجه می‌شود

بنابراین UCS از BFS هوشمندتر است، اما همچنان برای مسائل بزرگ محدودیت دارد.

7. الگوریتم A^* و طراحی heuristic

7.1 ایده‌ی کلی

A^* یکی از قدرتمندترین الگوریتم‌های جستجو است که از ترکیب دو مؤلفه استفاده می‌کند:

$$f(n) = g(n) + h(n) \quad f(n) = g(n) + h(n) \quad f(n) = g(n) + h(n)$$

که در آن:

- $g(n)$: هزینه‌ی طی شده تا حالت فعلی
- $h(n)$: تخمین هزینه‌ی باقیمانده تا هدف

اگر heuristic به‌خوبی طراحی شده باشد، A^* می‌تواند تعداد گره‌های بسط‌یافته را به‌طور قابل‌توجهی کاهش دهد.

7.2 طراحی heuristic در Sokoban

در Sokoban طراحی heuristic مناسب بسیار مهم است، زیرا heuristic ضعیف منجر به بررسی تعداد زیادی حالت می‌شود. در این پروژه، heuristic باید حداقل فاصله‌ی تقریبی هر جعبه تا نزدیک‌ترین هدف را تخمین بزند و تا حد امکان از مسیرهای بی‌ثمر جلوگیری کند.

یک heuristic ساده اما مؤثر می‌تواند مبتنی بر موارد زیر باشد:

- فاصله‌ی Manhattan بین جعبه‌ها و اهداف
- جفت کردن هر جعبه با نزدیک‌ترین هدف
- اضافه کردن جریمه برای وضعیت‌های پرخطر
- در نظر گرفتن بن‌بست‌های آشکار

7.3 اثر heuristic قوی

هرچه heuristic دقیق‌تر باشد، A^* هوشمندتر عمل می‌کند و states کمتری را expand می‌کند.

در نتیجه، نسبت به BFS و UCS، احتمالاً Expanded Nodes بسیار پایین‌تری خواهد داشت.

7.4 deadlock pruning در A^*

در A^* ، حذف حالت‌هایی که از ابتدا بن‌بست هستند اهمیت بسیار زیادی دارد. چون A^* حالت‌های promising را نگه می‌دارد، اگر deadlock‌ها حذف نشوند، حتی این الگوریتم نیز ممکن است در فضای حالت غرق شود.

در پیاده‌سازی این پروژه، ترکیب heuristic بهتر و تشخیص بن‌بست باعث شده A^* بهترین عملکرد را در میان الگوریتم‌ها داشته باشد.

7.5 مزایا

- یافتن جواب کم‌هزینه
- کاهش قابل‌توجه Expanded Nodes
- مناسب برای نقشه‌های متوسط و سخت
- قابل بهبود با heuristic بهتر

7.6 محدودیت‌ها

- کیفیت عملکرد کاملاً وابسته به heuristic است
- اگر heuristic ضعیف باشد، A^* به UCS نزدیک می‌شود
- طراحی heuristic خوب برای Sokoban کار ساده‌ای نیست

8. بررسی تجربی عملکرد الگوریتم‌ها در سطوح مختلف

در ارزیابی پروژه، معیار اصلی مقایسه، تعداد گره‌های بسط‌یافته است.

این معیار نشان می‌دهد هر الگوریتم برای رسیدن به جواب، چه مقدار از فضای حالت را واقعاً بررسی کرده است.

8.1 سطح آسان

در نقشه‌های آسان، معمولاً تعداد جعبه‌ها کمتر است و موانع پیچیده وجود ندارد. در این حالت، BFS و IDS نیز می‌توانند جواب را پیدا کنند، اما همچنان A^* و UCS از نظر کارایی بهتر عمل می‌کنند.

در سطح آسان:

- BFS معمولاً جواب را پیدا می‌کند ولی ممکن است حالت‌های زیادی را بسط دهد
- IDS به دلیل جستجوی تکراری، می‌تواند گره‌های بیشتری expand کند
- UCS مسیر کم‌هزینه‌تری نسبت به BFS ارائه می‌دهد
- A^* با heuristic مناسب کمترین تعداد گره بسط‌یافته را دارد

8.2 سطح متوسط

در نقشه‌های متوسط، تفاوت عملکرد الگوریتم‌ها بیشتر نمایان می‌شود.

اینجا BFS معمولاً با رشد شدید فضای حالت روبه‌رو می‌شود. IDS نیز به دلیل تکرار جستجو هزینه‌ی زیادی دارد. UCS بهتر از BFS است، اما هنوز بدون heuristic با حجم بالایی از حالت‌ها مواجه می‌شود.

در این سطح، A^* معمولاً برتری واضحی دارد، به خصوص اگر:

- heuristic مناسب باشد
- deadlock detection فعال باشد
- state hashing به درستی انجام شده باشد

8.3 سطح سخت

در نقشه‌های سخت، اهمیت بهینه‌سازی‌ها بسیار بیشتر می‌شود.

در این شرایط، BFS و IDS اغلب کارایی مناسبی ندارند و ممکن است زمان یا حافظه‌ی زیادی مصرف کنند. UCS نیز اگرچه مسیر کم‌هزینه پیدا می‌کند، اما ممکن است states زیادی را expand کند.

A^* در این سطح بیشترین سود را از heuristic و pruning می‌برد. به‌ویژه اگر حالت‌های بن‌بست حذف شوند، تعداد Expanded Nodes به شکل محسوسی کاهش می‌یابد.

8.4 جمع‌بندی تجربی

به‌طور کلی، الگوی عملکرد الگوریتم‌ها در این پروژه به صورت زیر است:

- BFS: ساده ولی پرهزینه از نظر فضای حالت
- IDS: کم‌مصرف از نظر حافظه، اما با تکرار زیاد
- UCS: مناسب برای هزینه‌های ناهمسان، ولی هنوز سنگین
- A^* : بهترین عملکرد در صورت طراحی heuristic مناسب و pruning مؤثر

9. نتیجه‌گیری نهایی

در این پروژه، مسئله‌ی Sokoban به عنوان یک مسئله‌ی جستجو در فضای حالت مدل‌سازی شد و چهار الگوریتم BFS، IDS، UCS و A^* برای حل آن بررسی و پیاده‌سازی شدند. همچنین با توجه به ماهیت مسئله، تکنیک‌های بهینه‌سازی مانند deadlock detection و حذف حالت‌های بی‌فایده نقش بسیار مهمی در بهبود عملکرد ایفا کردند.

نتایج نشان دادند که:

- الگوریتم‌های ناآگاهانه مانند BFS و IDS برای مسائل کوچک قابل استفاده‌اند، اما در نقشه‌های بزرگتر دچار رشد شدید فضای حالت می‌شوند.
- UCS با توجه به مدل هزینه‌ی مسئله، از BFS مناسب‌تر است.
- A^* با heuristic مناسب و pruning مؤثر، بهترین عملکرد را از نظر Expanded Nodes ارائه می‌دهد.

در نهایت می‌توان گفت که مهم‌ترین عامل موفقیت در این پروژه، فقط انتخاب الگوریتم نبود؛ بلکه طراحی مناسب state representation، hashing، heuristic و deadlock detection نیز نقش تعیین‌کننده‌ای در کاهش فضای جستجو و بهبود کارایی داشتند.